SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I YADDALA KUSHAL KUMAR(192524068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: YADDALA KUSHAL KUMAR

Course Faculty

Annexure A

DESIGN TASK

Q1. Predictive Parser Design (Application Level)

Task:
Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.

.

Grammar:
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → (E) | id

Conditions:

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

Test Case:
Input String:
id + id * id

Expected Output:

- Step-by-step parsing actions
- Final result: **String Accepted**

Yaddala Kushal Kumar
192524068

CO-2 Assessment tool-1

1.) Predictive parser Design LL(1)

Grammar

$E \to TE'$
$E' \to +TE' / \varepsilon$
$T \to FT'$
$T' \to *FT' / \varepsilon$
$F \to (E) / id$

First sets:

First(E) = {(, id}
First(E') = {+, ε}
First(T) = {(, id}
First(T') = {*, ε}
First(F) = {(, id}

Follow sets:

Follow(E) = {), $}
Follow(E') = {), $}
Follow(T) = {+,), $}
Follow(T') = {+,), $}
Follow(F) = {*, +,), $}

Predictive parsing table

Non-terminal	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \varepsilon$	$T' \to *FT'$		$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to id$			$F \to (E)$		

Stack	Input	Parsing Action
$E	id + id * id $	$E \rightarrow TE'$
$E'T	+ id * id $	$T \rightarrow FT'$
$E'T'F	+ id * id $	$F \rightarrow id$
$E'T'id	+ id * id $	$T' \rightarrow \varepsilon$
$E'T'	* id $	$E' \rightarrow +TE'$
$E'T'F*	id $	$T \rightarrow FT'$
$E'T'id	$	$F \rightarrow id$
$E'T'	$	$T' \rightarrow *FT'$
$E'	$	$F \rightarrow id$
$	$	$T' \rightarrow \varepsilon$
		$E' \rightarrow \varepsilon$
		Accept.

2.) Shift Reducing Parsing

Grammar:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

Q2. Shift-Reduce Parsing (Application Level)
Task:
Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:
E → E + E
E → E * E
E → id
Conditions:
- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

Test Case:
Input:
id + id * id
Expected Output:
- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Integrated Design Question (SLR, LALR, CLR Parsing)

2.)

Given that

Shift Reduce Parsing:-

Grammer:-

$E \rightarrow E+E$

$E \rightarrow E*E$

$E \rightarrow id$

Stack	Input	Action
$	id+id*id$	Shift
$ id	+id*id$	Reduce E→id
$ E	+id*id$	Shift
$ E+	id*id$	Shift
$ E+id	*id$	Reduce E→id
$ E+E	*id$	Reduce E→E+E
$ E	*id$	Shift
$ E*	id$	Shift
$ E*id	$	Reduce E→id
$ E*E	$	Reduce E→E*E
$ E	$	Accept

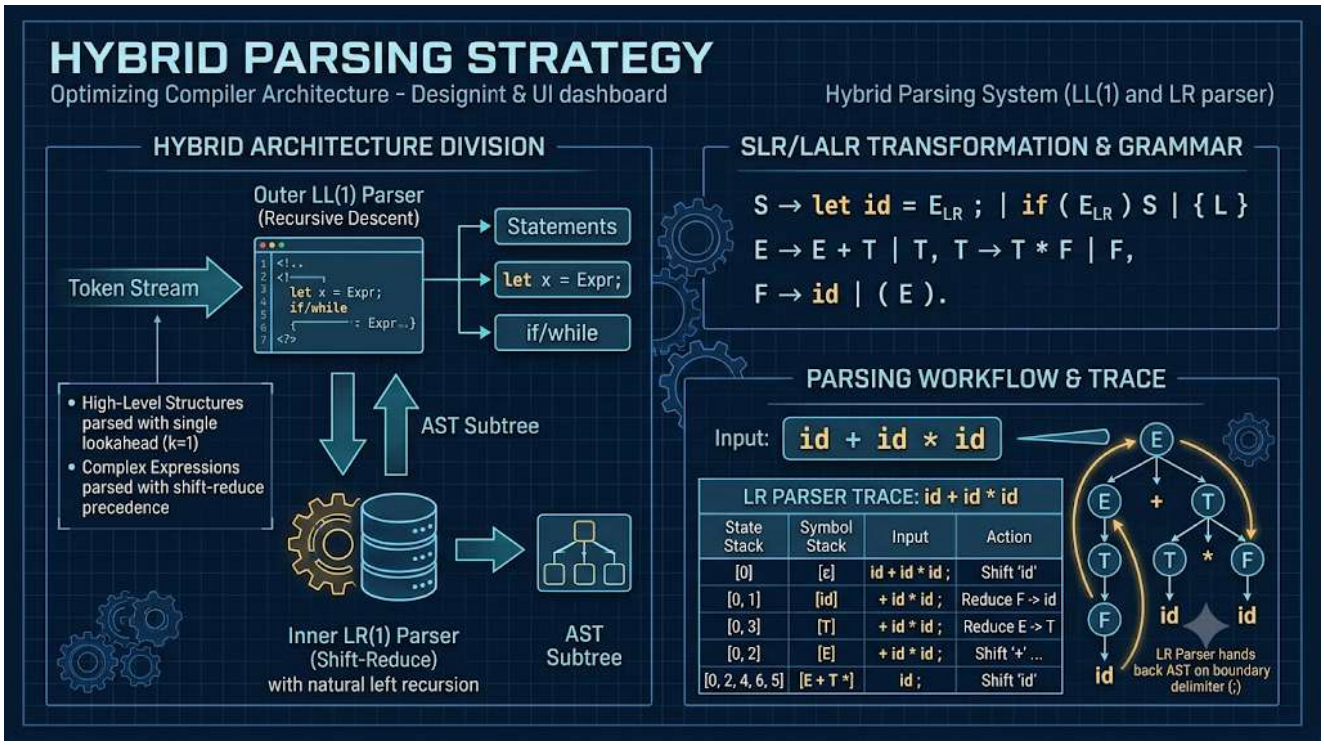
Q10. Design a Hybrid Parsing Strategy

Task:
Design a hybrid parsing system that combines LL(1) and LR parsing techniques to efficiently parse a programming language with both simple and complex constructs.

Conditions:

- Identify which parts use LL(1) vs LR parsing
- Justify the hybrid design choice
- Define grammar transformations required
- Illustrate parsing workflow with diagram
- Demonstrate parsing of a sample input
- Evaluate advantages over standalone methods

Test Case:
Input: id + id * id



Q11. Create an Unambiguous Grammar with Custom Operators

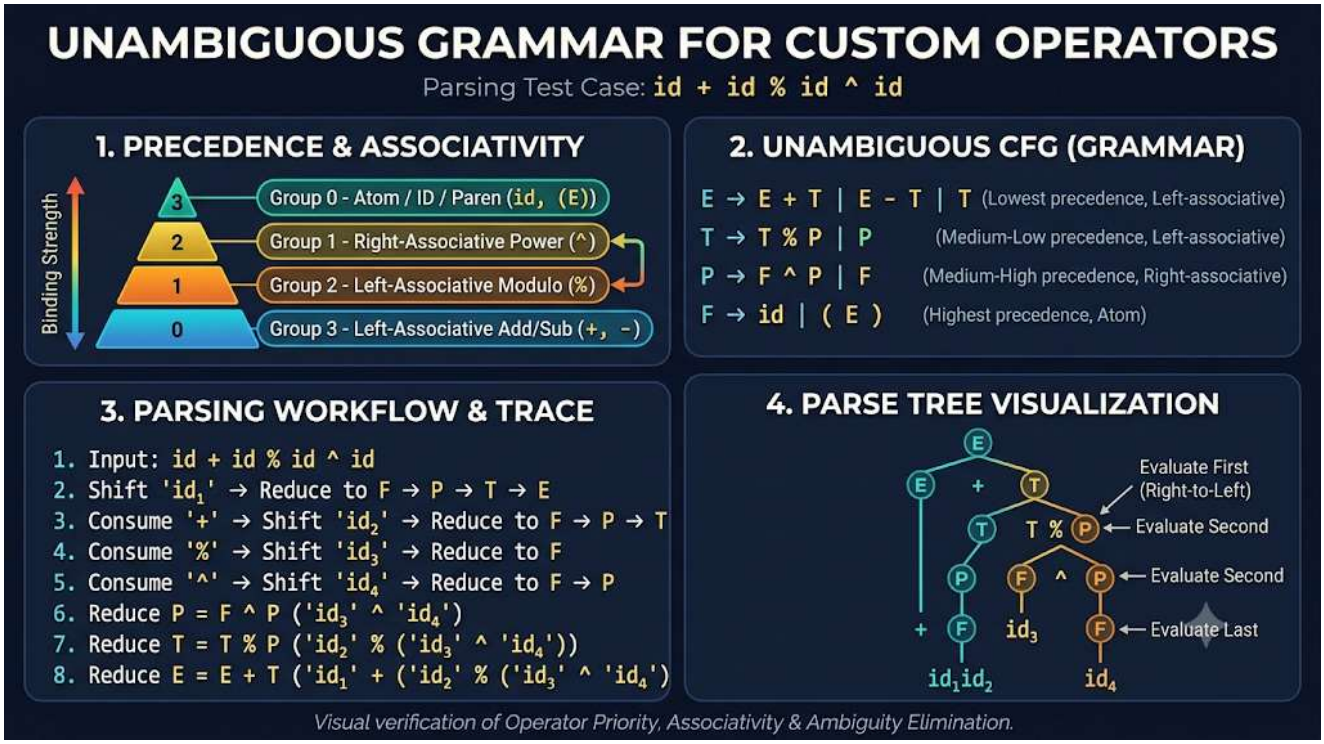
Task:
Design an unambiguous CFG for a language supporting custom operators (%, ^, +, -) with user-defined precedence and associativity, and construct a suitable parser.

Conditions:

- Define precedence hierarchy
- Handle both left and right associativity
- Eliminate ambiguity completely
- Construct parsing table (LL or LR)
- Validate grammar correctness

- Demonstrate parsing with detailed steps

Test Case:
Input: id + id % id ^ id



Q12. Develop a New Parsing Algorithm

Task:
Propose and design a new parsing algorithm that improves efficiency or readability compared to existing LL and LR techniques.

Conditions:

- Define algorithm steps clearly
- Compare with LL and LR methods
- Analyze time and space complexity
- Demonstrate parsing on sample grammar
- Highlight improvements and trade-offs
- Provide pseudo-code implementation

Test Case:
Input: id * id + id

THE EPD PARSING ALGORITHM (Eager Precedence Descent)

A Top-Down Parsing Approach with Dynamic Precedence & No State Tables | Parsing: $id_1 * id_2 + id_3$

1. COMPARATIVE PARSING MODELS



LL(k)
Poor with left recursion, deep nested ASTs.

LR(k)
Natively processes left recursion, massive tables, opaque state errors.

EPD (Proposed)
Dynamic numeric priority checks, $O(N)$ speed, lightweight recursive flow.

2. ALGORITHM LOGIC (EPD)

```
def parse_expression(min_precedence):
    left = parse_primary()  # Reads variables (id)
    while True:
        op = peek_token()
        if op is not operator or precedence(op) < min_precedence:
            break
        consume()
        # Compute next precedence dynamically based on associativity:
        next_min = precedence(op) + (1 if Left-Assoc else 0)
        right = parse_expression(next_min)
        left = ASTNode(op, left, right)
    return left
```

3. STEP-BY-STEP PARSING TRACE ($id_1 * id_2 + id_3$)

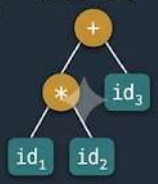
$id_1 * id_2 + id_3$			
Step	**Action**	**Active Stack / Call Level**	**Visual AST / Output State**
Step 1	`parse_primary()`	min_precedence = 0	Read id_1 -> (left = id_1)
Step 2	`peek_token()` -> '*'	precedence('*') = 2 >= 0	Keep processing. op = '*'. Next min = 3.
Step 3	`parse_expression(3)`	Deepen stack level	Reads id_2 -> (right = id_2)
Step 4	`peek_token()` -> '+'	precedence('+') = 1 < 3	True. Return subtree (id_2). Stack unwinds.
Step 5	Collapse AST	left = (id_1 * id_2)	Returns to base call level (min_precedence = 0)
Step 6	`peek_token()` -> '+'	precedence('+') = 1 >= 0	Keep processing. op = '+'. Next min = 2.
Step 7	`parse_expression(2)`	Deepen stack level	Reads id_3 -> (right = id_3)
Step 8	EOF / Complete	Collapse AST	Return final root: (+ (* id_1 id_2) id_3)

4. THE DYNAMIC PRECEDENCE RUNTIME

Token stream → id_1 * id_2 + id_3 → Consumed...

- Level 0: Root evaluation begins: `parse_expression(0)`
- The parser inspects * (Priority 2) and descends eager recursively: `parse_expression(3)` to gather id_2.
- Evaluates lookahead + (Priority 1) against the current threshold 3. Since 1 < 3, it pops/returns id_2, collapses the * tree, and ascends.
- High-level operator + (Priority 1) gets swallowed back at `parse_expression(0)`, which desolves resolves + as the new root.

Output Node Tree
(Visual AST)



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation